

Mantenimiento de Software

Apuntes de Clase

Este material reúne conceptos relacionados con mantenimiento de software, tomados de diferentes fuentes bibliográficas académicas.

¿Qué es el mantenimiento de software?

Según la terminología ANSI-IEEE, el mantenimiento del software es: *“la modificación de un producto software después de su entrega al cliente o usuario para corregir defectos, para mejorar el rendimiento u otras propiedades deseables, o para adaptarlo a un cambio de entorno”*.

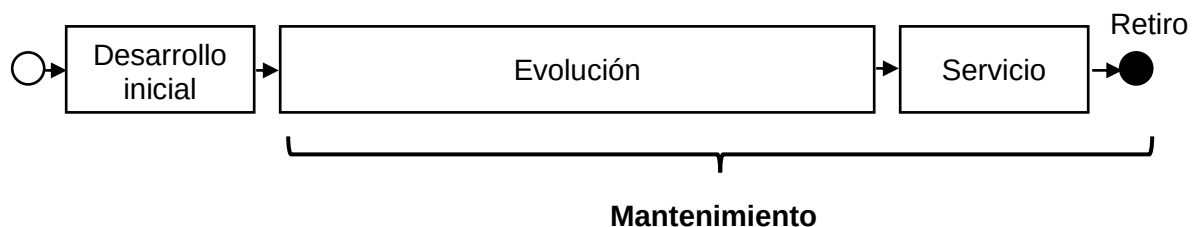
Presentación “Mantenimiento del Software”, Francisco Ruiz y Macario Polo. Grupo Alarcos, Universidad de Castilla-La Mancha, 2001. Página 6.

Disponible en Internet: <http://alarcos.esi.uclm.es/per/fruiz/curs/mso/trans/s1.pdf>

Hace algunos años, cuando se usaban metodologías tradicionales, el mantenimiento por lo general estaba al final del proceso de desarrollo, después de entregar el producto desarrollado a los clientes; sin embargo, con el uso de las metodologías ágiles, es posible hacer cambios en el software durante el mismo proceso de desarrollo, pues a medida que se hacen entregas parciales a los clientes, surgen recomendaciones, correcciones o extensiones que se deben realizar. Por lo tanto, para garantizar la calidad del software, muchas de las técnicas del mantenimiento han pasado a ser parte del proceso de desarrollo y es muy importante conocerlas y aplicarlas.

Fases

La siguiente imagen presenta las diferentes fases del ciclo de vida del software, donde se puede ver que la mayor parte del tiempo se dedica al mantenimiento. El mantenimiento, a su vez, tiene tres fases: evolución, servicio (u obsolescencia) y retiro:



Durante la evolución, el software se usa con éxito y hay un flujo constante de propuestas de cambios de requerimientos. Sin embargo, conforme el software se modifica, su estructura tiende a degradarse y los cambios se vuelven más y más costosos. (...) En dicha fase, el software avanza de la evolución al servicio. Durante la fase de servicio u obsolescencia, el software todavía es útil y se utiliza, pero sólo se le

realizan pequeños cambios tácticos. Durante esta fase, la compañía normalmente considera cómo reemplazar el software. En fase final, de retiro gradual (*Phase-out*), el software todavía puede usarse, aunque no se implementan más cambios.

Libro: "Ingeniería de Software", 9 Edición. Ian Sommerville. Editorial Pearson Educación, 2011, páginas 236 y 237.

Este libro se encuentra disponible en la base de datos "ebooks7-24",
<http://ezproxy.ucaldas.edu.co/login?url=http://www.ebooks7-24.com> (se ingresa con el correo institucional y luego se busca la palabra "software").

Importancia y costo del mantenimiento

La evolución del software es importante porque las organizaciones invierten grandes cantidades de dinero en él y en la actualidad son completamente dependientes de dichos sistemas. Sus sistemas se consideran activos empresariales críticos, por lo que tienen que invertir en el cambio del sistema para mantener el valor de estos activos. En consecuencia, las compañías más grandes gastan más en conservar los sistemas existentes que en el desarrollo de sistemas nuevos. Diferentes estudios indican que los costos del mantenimiento pueden estar entre el 50 % y el 90 % del costo total del software.

Libro "Ingeniería de Software", 9 Edición. Ian Sommerville. Editorial Pearson Educación, 2011, páginas 235 y 245.

Este libro se encuentra disponible en la base de datos "ebooks7-24",
<http://ezproxy.ucaldas.edu.co/login?url=http://www.ebooks7-24.com> (se ingresa con el correo institucional y luego se busca la palabra "software").

Acaso el lector pregunte por qué se requiere tanto mantenimiento y por qué se emplea tanto esfuerzo. Osborne y Chikofsky [Osb90] proporcionan una respuesta parcial:

Mucho del software del que dependemos en la actualidad tiene en promedio una antigüedad de 10 a 15 años. Aun cuando dichos programas se crearon usando las mejores técnicas de diseño y codificación conocidas en la época [y muchas no lo fueron], se produjeron cuando el tamaño del programa y el espacio de almacenamiento eran las preocupaciones principales. Luego migraron a nuevas plataformas, se ajustaron para cambios en máquina y tecnología de sistema operativo, y aumentaron para satisfacer las necesidades de los nuevos usuarios, todo sin suficiente preocupación por la arquitectura global. El resultado es estructuras pobremente diseñadas, pobre codificación, pobre lógica y documentación de los sistemas de software que ahora debemos seguir usando...

Otra razón del problema del mantenimiento del software es la movilidad del personal. Es probable que el equipo (o la persona) que hizo el trabajo original, o quienes hicieron alguna modificación, ya no estén más por ahí y, por lo tanto, nadie tiene conocimiento del sistema.

Libro "Ingeniería del Software: Un enfoque práctico", Séptima Edición. Roger S. Pressman. Editorial McGraw Hill, 2010, página 656.
Este libro se encuentra disponible en la base de datos "ebooks7-24",
<http://ezproxy.ucaldas.edu.co/login?url=http://www.ebooks7-24.com> (se ingresa con el correo institucional y luego se busca la palabra "software").

Tipos de mantenimiento

- *Mantenimiento correctivo*: tiene por objetivo **localizar y eliminar los defectos** o problemas encontrados de los programas. Dentro de esta categoría se incluye el mantenimiento de emergencia o urgente, el cual es un mantenimiento no planeado que se realiza para mantener temporalmente el software en funcionamiento, por algún error que bloquea el programa o los procesos de la empresa.
- *Mantenimiento adaptativo*: modificación de un programa debido a **cambios en el entorno** (hardware o software) en el cual se ejecuta. Los cambios pueden afectar, por ejemplo: el sistema operativo (actualización), la arquitectura física del sistema informático (paso de una arquitectura de red de área local a Internet/Intranet) o el entorno de desarrollo del software (incorporación de herramientas como ODBC).
- *Mantenimiento perfectivo*: conjunto de actividades para **mejorar o añadir nuevas funcionalidades** requeridas por el usuario. Se consideran dentro de este tipo de mantenimiento la incorporación de nuevas funcionalidades (mantenimiento de ampliación) y también los cambios que mejoran la eficiencia de ejecución del programa (mantenimiento de eficiencia).
Este tipo de mantenimiento es el más habitual.
- *Mantenimiento preventivo*: modificación del software para **mejorar las propiedades** de dicho software (por ejemplo, aumentando su calidad y/o su mantenibilidad) sin alterar sus especificaciones funcionales. Este tipo de mantenimiento utiliza en gran medida las técnicas de reingeniería.

Presentación "Mantenimiento del Software", Francisco Ruiz y Macario Polo. Grupo Alarcos, Universidad de Castilla-La Mancha, 2001. Página 19 a 26.
Disponible en Internet: <http://alarcos.esi.uclm.es/per/fruiz/curs/mso/trans/s1.pdf>

Actividades del mantenimiento de software

Las actividades del mantenimiento de software se pueden agrupar en tres categorías:

- *Comprensión del software y de los cambios a realizar*: para poder modificar un programa, los programadores necesitan conocer su funcionalidad y objetivos, su estructura interna y los requisitos de operación. De no ser así, se corre un gran

riesgo de introducir nuevos defectos que en el futuro supondrán un coste de mantenimiento adicional. Estas actividades incluyen estudiar las peticiones, estudiar la documentación y estudiar el código.

Dentro de estas actividades también se pueden incluir: determinar el impacto del cambio, estudiar alternativas de solución y hacer la planeación.

Este grupo de actividades es el que toma más tiempo durante un mantenimiento, sobre todo cuando el software no está bien documentado y/o no tiene un buen diseño.

- *Modificación del software:* para incorporar los cambios necesarios se deben crear y modificar el diseño, las estructuras de datos, la lógica de los procesos y las interfaces. Los programadores deben conocer lo mejor posible las repercusiones que tienen en el sistema los cambios que están realizando, con el fin de evitar al máximo posible los efectos secundarios. Estas actividades incluyen hacer el diseño (o rediseño), modificar el código y actualizar las versiones.
- *Realización de pruebas:* para validar los cambios se deben realizar pruebas selectivas que nos permitan comprobar la corrección del software. Esta actividad es necesaria siempre, ya que incluso un cambio muy pequeño no verificado puede producir defectos en el software que reduzcan su calidad y fiabilidad. Se incluye en esta categoría el diseño de las pruebas y su ejecución, y también se incluye la actualización de la documentación (de diseño, código y pruebas).

Presentación "Mantenimiento del Software", Francisco Ruiz y Macario Polo. Grupo Alarcos, Universidad de Castilla-La Mancha, 2001. Páginas 28 y 29.

Disponible en Internet: <http://alarcos.esi.uclm.es/per/fruiz/curs/mso/trans/s1.pdf>

Además de las actividades, en el mantenimiento de software en ocasiones se usan algunas técnicas que son complementarias al desarrollo de software. Dentro de estas técnicas se encuentran:

- Comprensión del programa
- Reingeniería (incluyendo ingeniería inversa, reestructuración e ingeniería directa)
- Migración

Libro: "Guide to the software engineering body of knowledge (SWEBOK (R)): Version 3.0". Editado por BOURQUE, Pierre, et al. IEEE Computer Society Press, 2014, páginas 5-8 a 5-10.

Disponible en Internet (previo registro) en:

<https://www.computer.org/education/bodies-of-knowledge/software-engineering/v3>

Mantenibilidad

Esta característica representa la capacidad del producto software para ser modificado efectiva y eficientemente, debido a necesidades evolutivas, correctivas o perfectivas.

Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- Modularidad. Capacidad de un producto para evitar que los cambios en un componente afecten a otros componentes.
- Reusabilidad. Capacidad de un activo que permite que sea utilizado en más de un sistema software o en la construcción de otros activos.
- “Analizabilidad”. Facilidad con la que se puede evaluar el impacto de un determinado cambio sobre el resto del software, diagnosticar las deficiencias o causas de fallos en el software, o identificar las partes a modificar.
- Capacidad para ser modificado. Capacidad del producto que permite que sea modificado de forma efectiva y eficiente sin introducir defectos o degradar el desempeño.
- Capacidad para ser probado. Facilidad con la que se pueden establecer criterios de prueba para un sistema o componente y con la que se pueden llevar a cabo las pruebas para determinar si se cumplen dichos criterios.

Norma ISO/IEC 25010 - *System and software quality models*, 2011. Disponible en Internet:
<https://iso25000.com/index.php/normas-iso-25000/iso-25010>